TUGAS PERTEMUAN: 4

JAVASCRIPT

NAMA/NIM	:	Rahman Mauliddin Anwar (2618903)
KELAS	:	D
PENGAJAR	:	MUHAMMAD FAUZUL ADHIM (2418046)

4.1 Tujuan

1. Praktikan mampu memahami dasar-dasar *Javascript*.
2. Praktikan mampu memahami penggunaan *Javascript*.
3. Praktikan mampu memahami DOM pada *JavaScript*, termasuk *popup box* dan *event*.

4.2 Alat dan bahan

1. Laptop
2. *Visual Studio Code*
3. Modul Praktikum Web 2026

4.3 Langkah-langkah

1. Hubungkan file tugas.js dengan file index.html dan login html dengan menambahkan tag *script* tepat di bawah js/app.js.

   Source Code :

```
<script src="js/app.js"></script>
<script src="js/tugas.js"></script>
```

2. Pada tugas.js, tulis bagian awal yang berisi CSS tambahan, fungsi showToast(), pembungkus *localStorage* (*Web Storage*), dan fungsi *delay*() yang mengembalikan *Promise* seperti berikut ini.

   Source Code :

```
(function () {
  "use strict";
  const style = document.createElement("style");
  style.textContent = `
    .tg-toast {
      position: fixed; bottom: 24px; left: 50%;
      transform: translateX(-50%);
      background: #333; color: #fff;
      padding: 12px 20px; border-radius: 8px;
      font-size: 14px; z-index: 9999;
      box-shadow: 0 4px 12px rgba(0,0,0,.25);
      opacity: 0; transition: opacity .3s;
      max-width: 90vw; text-align: center;
    }
    .tg-toast.show { opacity: 1; }
```

```
    .tg-extra { margin: 10px 0 0 1.1rem; display: flex; flex-wrap:
wrap; align-items: center; gap: 12px; }
    .tg-extra .tg-sapaan { margin: 0 !important; font-size: 16px
!important; color: #1f48ff; font-weight: 700; }
    .tg-extra .tg-btn {
      border: 2px solid #1f48ff; background: #fff; color: #1f48ff;
      padding: 6px 14px; border-radius: 20px; font-size: 13px;
      font-weight: 600; cursor: pointer; transition: .2s;
    }
    .tg-extra .tg-btn:hover { background: #1f48ff; color: #fff; }

    .tg-quote {
      position: fixed; right: 20px; bottom: 20px; z-index: 50;
      width: 300px; max-width: calc(100vw - 40px);
      background: #fff; border-left: 5px solid #1f48ff; border-
radius: 10px;
      padding:    14px    16px;    box-shadow:    0    6px    20px
rgba(4,21,122,.18);
      font-size: 13px; line-height: 1.5; color: #1a2038;
    }
    .tg-quote  h4  {  margin:  0  0  6px;  font-size:  13px;  color:
#04157a; }
    .tg-quote .tg-q-text { margin: 0; font-style: italic; }
    .tg-quote .tg-q-by { margin: 4px 0 8px; color: #777; font-
size: 12px; }
    .tg-quote .tg-btn {
      border: 0; background: #1f48ff; color: #fff; padding: 5px
12px;
      border-radius: 14px; font-size: 12px; cursor: pointer;
    }
    .tg-quote .tg-btn:disabled { opacity: .6; cursor: wait; }

    .tg-hint { font-size: 12px; margin: 4px 0 0; min-height: 14px;
}
    .tg-hint.ok { color: #2e7d32; }
    .tg-hint.bad { color: #e53935; }
    .tg-focus { outline: 2px solid #1f48ff; outline-offset: 2px;
border-radius: 8px; }
  `;
  document.head.append(style);
  function showToast(pesan) {
    const toast = document.createElement("div");   // DOM: membuat
elemen baru
    toast.className = "tg-toast";
    toast.textContent = pesan;
    document.body.append(toast);                  // DOM: menambah
elemen ke halaman
    setTimeout(()  =>  toast.classList.add("show"),  10);      //
setTimeout: tampilkan setelah 10 ms
    setTimeout(() => {                                          //
hilangkan setelah 3 detik
      toast.classList.remove("show");
      setTimeout(() => toast.remove(), 300);
    }, 3000);
  }
  const simpan = {
    // ambil data dari localStorage; jika belum ada, kembalikan
nilai awal
    ambil(key, awal) {
      try { return localStorage.getItem(key) ?? awal; } catch (e)
{ return awal; }
    },
```

```
    // simpan data ke localStorage (tetap ada walau browser
ditutup)
    taruh(key, nilai) {
      try { localStorage.setItem(key, nilai); } catch (e) { /*
abaikan */ }
    }
  };

  function delay(ms) {
    return new Promise(function (resolve) {
      setTimeout(resolve, ms);
    });
  }
```

3. Selanjutnya, tambahkan fungsi *landing*() pada halaman index.html. Fungsi *landing*() akan memanipulasi DOM, menangani event seperti *mouseenter, mouseleave, click, dblclick, keydown, contextmenu, offline, online,* dan menampilkan kotak *popup* (*prompt*, *confirm*, *alert*) seperti di bawah ini.

   Source Code :

```
  function landing() {

    document.getElementById("subJudul").innerHTML = "Raih Karier
Impianmu";

    const fitur = document.querySelectorAll(".feat");
    fitur.forEach(function (f) {
      f.addEventListener("mouseenter", function () {
        f.style.backgroundColor = "#eaf2f8";
      });
      f.addEventListener("mouseleave", function () {
        f.style.backgroundColor = "";
      });
    });

    const catatan = document.createElement("p");
    catatan.textContent = "Ribuan lowongan baru setiap hari!";
    catatan.style.fontWeight = "600";
    document.getElementById("deskripsi").after(catatan);

    fitur.forEach(function (f, i) {
      f.setAttribute("title", "Fitur ke-" + (i + 1));
    });

    const extra = document.createElement("div");
    extra.className = "tg-extra";
    extra.innerHTML =
      '<p class="tg-sapaan" id="tgSapaan"></p>' +
      '<button type="button" class="tg-btn" id="tgNama">Ubah Nama
Sapaan</button>';
    catatan.after(extra);

    const sapaan = document.getElementById("tgSapaan");

    function tampilSapaan() {
      const nama = simpan.ambil("tg_nama", "");
      sapaan.textContent = nama ? "Halo, " + nama + "!" : "";
    }
    tampilSapaan();

    const kunjungan = Number(simpan.ambil("tg_kunjungan", 0)) +
1;
```

```
    simpan.taruh("tg_kunjungan", kunjungan);

    const namaTersimpan = simpan.ambil("tg_nama", "");
    showToast(
      namaTersimpan
       ? "Selamat datang kembali, " + namaTersimpan + "! (kunjungan
ke-" + kunjungan + ")"
        : "Selamat datang di CarrerGo! (kunjungan ke-" + kunjungan
+ ")"
    );

      document.getElementById("tgNama").addEventListener("click",
function () {
          const input = prompt("Silakan masukkan namamu:",
simpan.ambil("tg_nama", ""));

      if (input === null) return;

      if (input.trim() === "") {
        alert("Nama tidak boleh kosong!");
        return;
      }
      simpan.taruh("tg_nama", input.trim());
      tampilSapaan();
      showToast("Nama sapaan disimpan: " + input.trim());
    });

    fitur.forEach(function (f) {
      f.addEventListener("click", function () {
         const namaFitur = f.textContent.trim().replace(/\s+/g, "
");
        // confirm() mengembalikan true (OK) atau false (Cancel)
          const lanjut = confirm(namaFitur + "\n\nIngin daftar
sekarang untuk memakai fitur ini?");
        if (lanjut) {
          location.href = "register.html";
        } else {
          showToast("Oke, silakan jelajahi dulu.");
        }
      });
    });

    document.querySelector(".brand").addEventListener("dblclick",
function (e) {
      e.preventDefault();
      alert("CarrerGo v1.0\nTugas Praktikum Pemrograman Web - BAB
4 JavaScript II");
    });

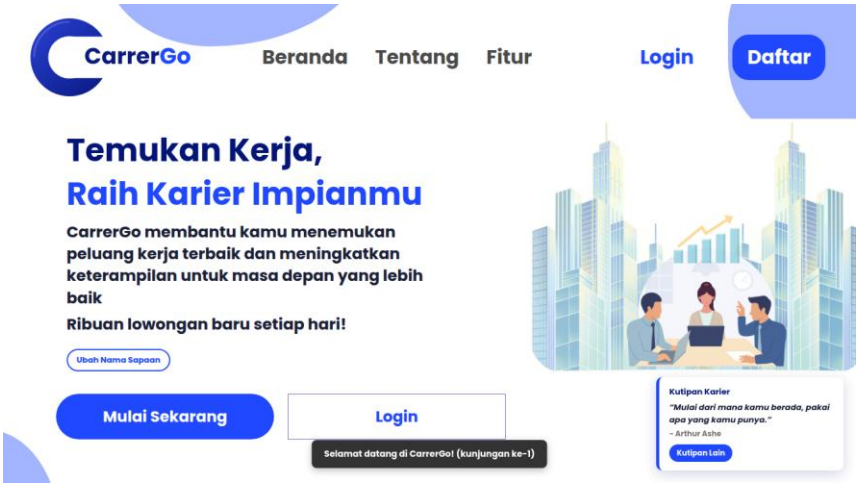
    document.addEventListener("keydown", function (e) {
      if (e.ctrlKey || e.altKey || e.metaKey) return;
      const tag = (e.target.tagName || "").toLowerCase();
      if (tag === "input" || tag === "textarea") return;
          if (e.key.toLowerCase() === "l") location.href =
"login.html";
          if (e.key.toLowerCase() === "d") location.href =
"register.html";
    });

                                    document.querySelector(".hero-
img").addEventListener("contextmenu", function () {
      showToast("Ilustrasi ini milik CarrerGo.");
    });
```

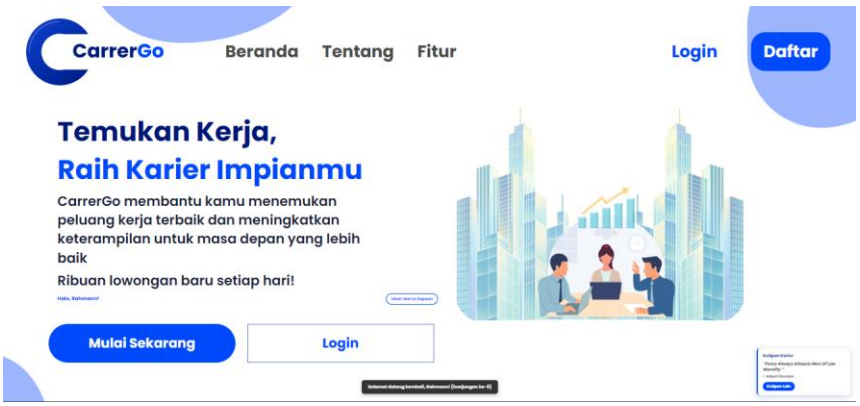
```
        window.addEventListener("offline",    function    ()    {
showToast("Kamu sedang offline."); });
         window.addEventListener("online",    function    ()    {
showToast("Kamu kembali online."); });

    buatKartuKutipan();
  }
```

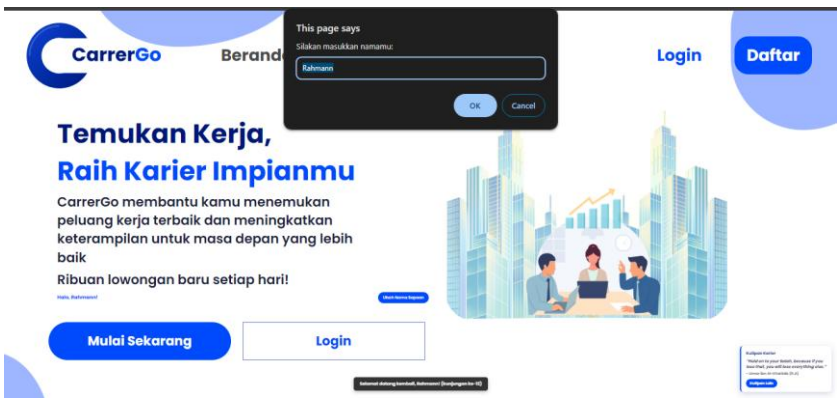
Hasil :



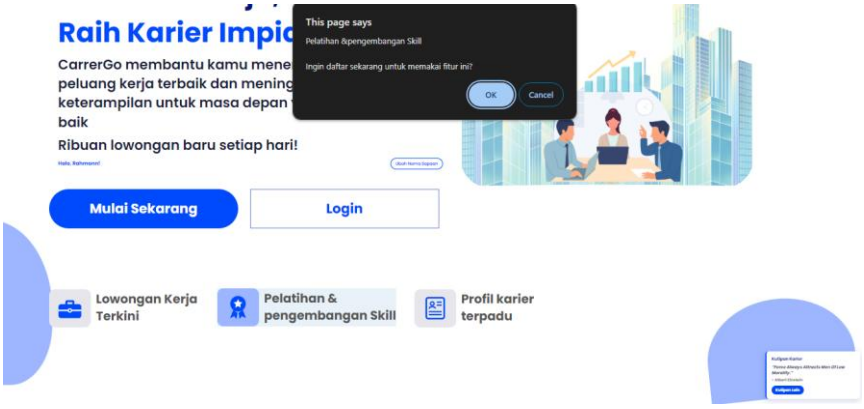
Gambar 4. 1 *Output* index.html saat pertama dibuka



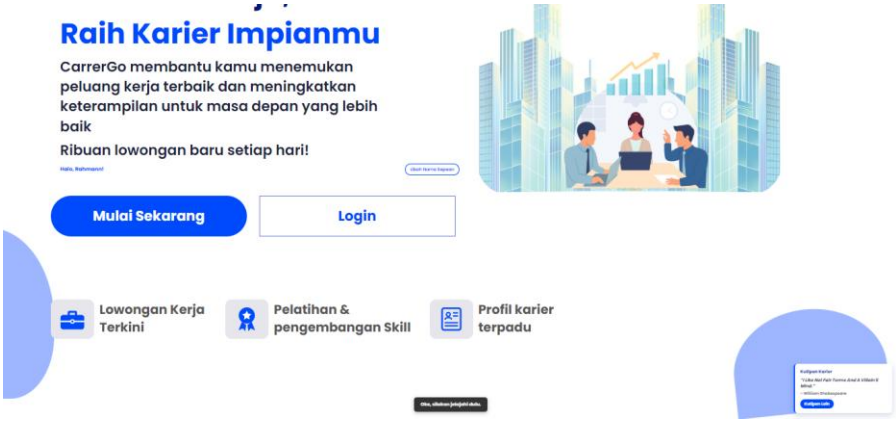
Gambar 4. 2 Sapaan tersimpan dan *event mouseenter* pada kartu fitur



Gambar 4. 3 *Popup box prompt*() untuk mengisi nama sapaan



Gambar 4. 4 *Popup box confirm*() saat kartu fitur diklik



Gambar 4. 5 Hasil setelah memilih *Cancel* pada confirm()

3. Selanjutnya, tambahkan fitur Kutipan Karier yang mengambil data dari internet menggunakan *async/await*, *Promise.all*, dan *fetch* seperti di bawah ini.

   Source Code:

```
const KUTIPAN_CADANGAN = [
    { quote: "Kesempatan datang pada mereka yang siap.",
author: "CarrerGo" },
    { quote: "Mulai dari mana kamu berada, pakai apa yang
kamu punya.", author: "Arthur Ashe" },
    { quote: "Karier terbaik dibangun dari langkah kecil
yang konsisten.", author: "CarrerGo" }
  ];

  // async function: dapat memakai await, sehingga kode
asynchronous
  // terbaca seperti kode biasa dan tidak memblokir
halaman.
  async function ambilKutipan() {
    // AbortController: membatalkan fetch bila server
lebih dari 5 detik
    const pembatal = new AbortController();
    const timer = setTimeout(() => pembatal.abort(),
5000);
```

```
    try {
      // Promise.all: fetch dan delay(500) dijalankan
bersamaan (paralel);
      // await menunggu keduanya selesai. delay membuat
teks "Memuat..." sempat terlihat.
      const [respon] = await Promise.all([
        // FETCH: mengambil data kutipan acak dari API
(mengembalikan Promise)
        fetch("https://dummyjson.com/quotes/random", {
signal: pembatal.signal }),
        delay(500)
      ]);

      if (!respon.ok) throw new Error("Respon server: " +
respon.status);
      return await respon.json();               // ubah
respons HTTP menjadi objek JSON
    } catch (err) {
      // Bila gagal (offline, timeout, error server) pakai
kutipan cadangan
      console.warn("Fetch gagal, memakai kutipan
cadangan:", err.message);
      return KUTIPAN_CADANGAN[Math.floor(Math.random() *
KUTIPAN_CADANGAN.length)];
    } finally {
      clearTimeout(timer);                          // selalu
hentikan timer
    }
  }

  // Membuat kartu kutipan di pojok kanan bawah halaman
  function buatKartuKutipan() {
    const kartu = document.createElement("aside");
    kartu.className = "tg-quote";
    kartu.innerHTML =
      "<h4>Kutipan Karier</h4>" +
      '<p class="tg-q-text" id="tgQ">Memuat...</p>' +
      '<p class="tg-q-by" id="tgBy"></p>' +
      '<button type="button" class="tg-btn"
id="tgQBtn">Kutipan Lain</button>';
    document.body.append(kartu);

    const teks = kartu.querySelector("#tgQ");
    const oleh = kartu.querySelector("#tgBy");
    const tombol = kartu.querySelector("#tgQBtn");

    // Memuat kutipan: nonaktifkan tombol -> tunggu data -
> tampilkan
    async function muat() {
      tombol.disabled = true;
      teks.textContent = "Memuat...";
      oleh.textContent = "";

      const data = await ambilKutipan();         // await:
tunggu sampai Promise selesai
      teks.textContent = "\u201C" + data.quote + "\u201D";
      oleh.textContent = "- " + data.author;
      tombol.disabled = false;
```

```
    }

    tombol.addEventListener("click", muat);       // EVENT
click: ambil kutipan baru
    muat();                                       // dimuat
otomatis saat halaman dibuka
  }
```

Hasil :



Gambar 4. 6 Kartu Kutipan Karier

4. Selanjutnya, tambahkan fungsi login() untuk halaman login.html yang mencakup *event input, change, focus, blur, keydown*, dan *keyup*, serta menyimpan *email* di *localStorage*.

   Source Code:

```
function login() {
const email = document.getElementById("email");
const pass = document.getElementById("password");

// WEB STORAGE: isi otomatis email yang terakhir dipakai
if (!email.value) email.value = simpan.ambil("tg_email",
"");

// EVENT change: simpan email ke localStorage ketika nilai
input berubah
email.addEventListener("change", function () {
simpan.taruh("tg_email", email.value.trim());
});

// Elemen petunjuk validasi email (dibuat lewat DOM)
const hint = document.createElement("p");
hint.className = "tg-hint";
email.closest(".field").append(hint);

// EVENT input: cek format email setiap kali pengguna
mengetik
email.addEventListener("input", function () {
const valid =
/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email.value.trim());
if (email.value.trim() === "") {
hint.textContent = "";
hint.className = "tg-hint";
} else if (valid) {
hint.textContent = "Format email sudah benar";
hint.className = "tg-hint ok";
} else {
hint.textContent = "Format email belum benar";
```

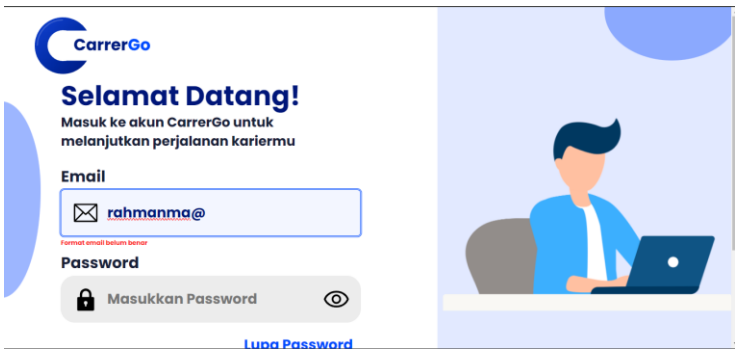
```
hint.className = "tg-hint bad";
}
});

// EVENT focus / blur: beri garis tepi saat kolom dipilih,
hapus saat ditinggalkan
[email, pass].forEach(function (el) {
el.addEventListener("focus", function () {
el.closest(".input-wrap").classList.add("tg-focus"); });
el.addEventListener("blur", function () {
el.closest(".input-wrap").classList.remove("tg-focus");
});
});

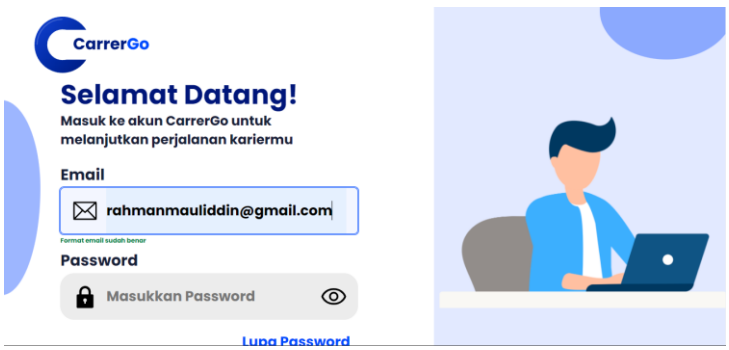
// EVENT keydown / keyup: peringatan bila Caps Lock aktif
saat mengetik password
const capsHint = document.createElement("p");
capsHint.className = "tg-hint bad";
pass.closest(".field").append(capsHint);

function cekCaps(e) {
capsHint.textContent = e.getModifierState &&
e.getModifierState("CapsLock")
? "Caps Lock sedang aktif" : "";
}
pass.addEventListener("keydown", cekCaps);
pass.addEventListener("keyup", cekCaps);
pass.addEventListener("blur", function () {
capsHint.textContent = ""; });
}
```

Hasil:



Gambar 4. 7 *Output* login.html saat format *email* belum benar



Gambar 4. 8 *Output* login.html saat format *email* sudah benar

5. Terakhir, tambahkan kode pemilih halaman di bagian paling bawah tugas.js agar fungsi yang dijalankan sesuai atribut *data-page* pada *body*.

   Source Code:

```
  const halaman = document.body.dataset.page;
  if (halaman === "landing") landing();
  if (halaman === "login") login();
})();
```

Analisa:

Praktikum ini menambahkan *JavaScript* ke *project CarrerGo* supaya halaman menjadi lebih interaktif. Semua kode ditempatkan di satu file, tugas.js, yang dimuat di index.html dan login.html. Kode dibungkus dalam IIFE dengan "*use strict*" agar semua variabel tidak bentrok. Atribut *data-page* pada *body* menentukan fungsi yang dijalankan: *landing*() untuk halaman utama dan *login*() untuk halaman masuk. DOM digunakan untuk mengubah halaman secara dinamis: *innerHTML* mengubah teks judul, *createElement*() dengan *after*() dan *append*() membuat elemen baru, *setAttribute*() menambah atribut *title*. *Event mouseenter* dan *mouseleave* mengubah warna kartu fitur, *contextmenu* menampilkan pesan ketika gambar diklik kanan, *offline* dan *online* memantau koneksi internet, dan keydown membuat tombol L membuka halaman *login* serta tombol D membuka halaman daftar.

Popup box yang digunakan mencakup tiga jenis. *prompt*() meminta nama sapaan dan menghentikan proses jika hasilnya *null* (tombol *Cancel*). *alert*() menampilkan peringatan ketika nama dikosongkan dan menampilkan informasi versi ketika logo diklik dua kali (*event dblclick*). *confirm*() bertanya apakah pengguna ingin mendaftar ketika kartu fitur diklik; jika *true*, pengguna diarahkan ke register.html, dan jika *false*, hanya muncul notifikasi. *Web Storage* digunakan lewat *localStorage* dengan objek simpan yang membungkus getItem() dan setItem() di dalam *try-catch*. Data yang disimpan meliputi nama sapaan, jumlah kunjungan, dan *email* terakhir, sehingga data tetap ada meskipun halaman dimuat ulang atau *browser* ditutup.

Fitur Kutipan Karier menunjukkan konsep *asynchronous*. Fungsi ambilKutipan() memakai *async/await* supaya halaman tidak terhenti; *fetch* mengambil kutipan acak dari *API dummyjson*, dan *delay*() yang mengembalikan Promise dijalankan bersamaan lewat *Promise.all* agar tulisan "Memuat…" muncul setidaknya 500 milidetik. *AbortController* membatalkan permintaan jika *server* tidak merespons dalam 5 detik, dan blok *try-catch-finally* memastikan kutipan cadangan dipakai ketika gagal serta timer selalu dihentikan. Di halaman *login*, event input memeriksa format email dengan regular expression dan menampilkan petunjuk hijau atau merah; event focus dan blur mengatur garis tepi kolom yang dipilih; serta event *keydown* dan *keyup* membaca *getModifierState*("*CapsLock*")

untuk memperingatkan ketika *Caps Lock* aktif. Dengan begitu, *JavaScript* mengatur perilaku halaman, sementara struktur tetap di HTML dan tampilan di CSS.